

Competitive Programming HandsOn N.2

Matteo Giuntoni

September 2026

1 The "Min Max" Problem

To solve this problem in the required time of $\mathcal{O}((n+m) \log n)$, where n is the length of the initial array and m is the number of queries, it is necessary to use a **Segment Tree** data structure, storing the maximum value present in the corresponding sub-array for each segment $[i, j]$ in the tree.

To strictly meet the required time constraints, the solution is structured into three phases: initializing the data structure through an **Insert** process, modifying ranges using an **Update** operation with *lazy propagation*, and retrieving the maximum value via a **Max** query.

First, let's define the fundamental structures and parameters underlying the Segment Tree:

```
pub struct Elem {
    pub val: i32,
    pub to_propagate: bool,
}
pub struct SegmentTree {
    tree: Vec<Elem>,
    max_val: usize,
}
```

Below is the description of the various parameters:

- **val**: stores the maximum value present in the segment corresponding to the node.
- **to_propagate**: boolean flag used for *lazy propagation*; indicates whether the node has a pending update that has yet to be applied (propagated) to its children.
- **tree**: the vector physically representing the tree, sized to contain the **Elem** instances.
- **max_val**: represents the maximum size n of the original array, used as an upper bound to correctly calculate the limits (start and end) during recursive navigation.

1. Insert

To use this structure, an initial construction phase is required, in which all the values of the starting array are inserted. This operation is performed using the

insert method. The method inserts the value val , originally located at index idx , into the Segment Tree.

The operation is carried out recursively by navigating the Segment Tree until reaching the leaf corresponding to the idx position, choosing the sub-tree (or sub-segment) of the current node by comparing idx with the mid value. During the navigation of the tree towards the leaf corresponding to idx , the method updates every intermediate node encountered: specifically, the value of the current segment is replaced with the maximum between itself and the new val .

Through this approach, each node in the Segment Tree will maintain the previously described structural invariant, always containing the maximum value of the reference sub-array.

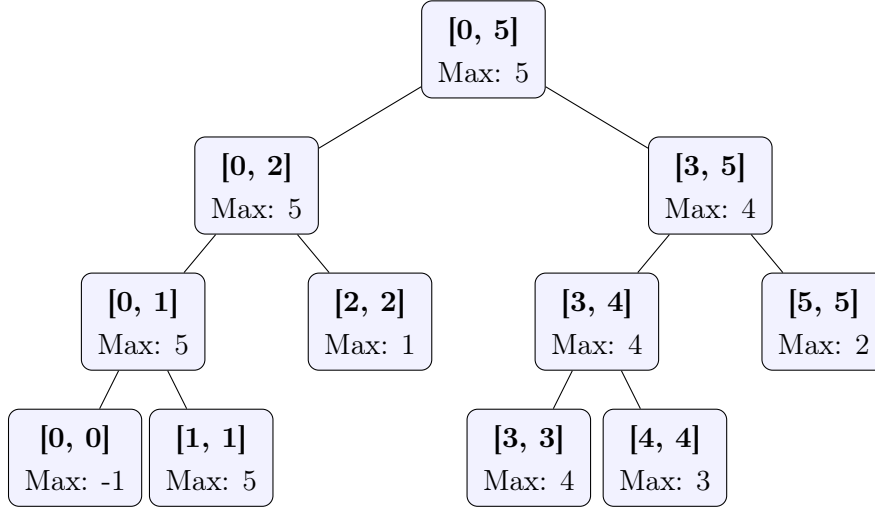


Figure 1: Representation of the Segment Tree after the Insert phase for the array $[5, 1, 4, 3, 2]$. Each node represents its interval of responsibility and the maximum value contained within it.

2. Update

This section describes the logic of the first operation required by the problem: updating a range (**Update**(i, j, T)). Since I am using a Segment Tree to store the maximum value for each segment, this operation has been translated into an update of the Segment Tree, maintaining the invariant defined above. To perform this operation while ensuring the target time of $\mathcal{O}(\log n)$, it is necessary to implement a recursive traversal of the Segment Tree.

During the descent through the tree, the recursion stops as soon as the interval handled by the current node ($[start, end]$) is completely contained within the query interval $[i, j]$. Upon reaching this base condition, we update the node's value by

calculating the minimum between its current value and the `val` parameter provided in the input, as requested.

```
if i <= start && j >= end {
    self.tree[node].val = cmp::min(val, self.tree[node].val);
    self.tree[node].to_propagate = true;
}
```

As noted in the code, in addition to updating the value, the `to_propagate` flag is activated. This step introduces the concept of **lazy propagation**.

Indeed, to keep the fundamental property of the Segment Tree valid (each node must contain the maximum of its sub-array), a naive approach would require propagating this update down to all underlying leaf nodes. This would result in unnecessary and costly work: in the case of extensive or consecutive updates, the time complexity would degrade towards $\mathcal{O}(n)$, compromising the algorithm's performance.

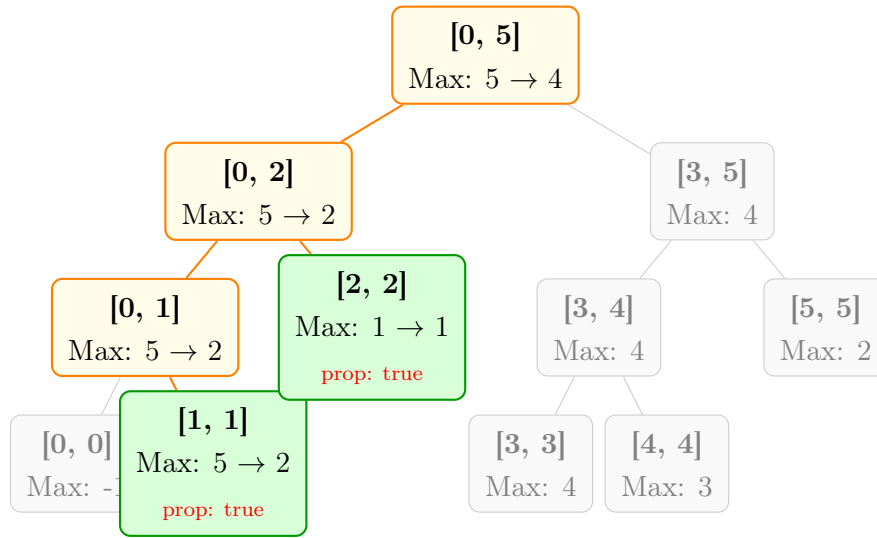


Figure 2: Execution trace of `Update(1, 2, 2)`. In yellow, visited nodes that partially overlap; in green, nodes where the interval is completely contained in the query range and recursion stops, setting the `to_propagate` flag. In gray, ignored nodes.

Lazy propagation allows updates to descendants to be postponed until it is strictly necessary to visit them. In subsequent operations, if the recursive traversal (whether for an `Update` or a `Max`) needs to descend into a node's children, its flag status is checked first. If `to_propagate` is active, the *push-down* phase occurs: the parent node's value is applied to the children, and the pending operation is passed to them. Once the current node is "cleaned" by deactivating its flag, the descent

can safely continue.

```
if self.tree[node].to_propagate {
  self.tree[node].to_propagate = false;

  self.tree[left_node].val = cmp::min(self.tree[node].val,
                                       self.tree[left_node].val);
  self.tree[left_node].to_propagate = true;

  self.tree[right_node].val = cmp::min(self.tree[node].val,
                                       self.tree[right_node].val);
  self.tree[right_node].to_propagate = true;
}
```

Finally, a crucial step of the algorithm occurs during the ascent back up the tree. Once the recursive calls on the children to update only the necessary portions of the tree are finished, the current parent node must be recalculated to reflect the new underlying maximums. It will then take the maximum between the updated value of the left branch and that of the right branch.

```
let left = if i <= mid {
  self.update_rec(left_node, start, mid, i, j, val)
} else {
  self.tree[left_node].val
};

let right = if j > mid {
  self.update_rec(right_node, mid + 1, end, i, j, val)
} else {
  self.tree[right_node].val
};

self.tree[node].val = cmp::max(left, right);
```

3. Max

This section analyzes the second operation required by the problem: finding the maximum value within a given range ($\text{Max}(i, j)$). As with the update, the goal is to answer this query in $\mathcal{O}(\log n)$ time, leveraging the previously constructed tree structure.

Given the properties of the Segment Tree, answering this query will require navigating the tree until identifying the segments that exactly cover the requested interval. The search proceeds via a top-down recursive traversal. The termination condition occurs when the interval handled by the current node ($[start, end]$) is entirely contained within the query interval $[i, j]$. In this case, the node already contains the correct maximum value for that specific portion of the array, and it is sufficient to return it without further exploring its children.

```
if i <= start && j >= end {
    return self.tree[node].val;
}
```

However, before being able to descend into the sub-trees for queries that only partially cover the current segment, it is essential to ensure that the read data is up-to-date. If the current node has a pending update from a previous **Update** operation, this must be propagated to its children.

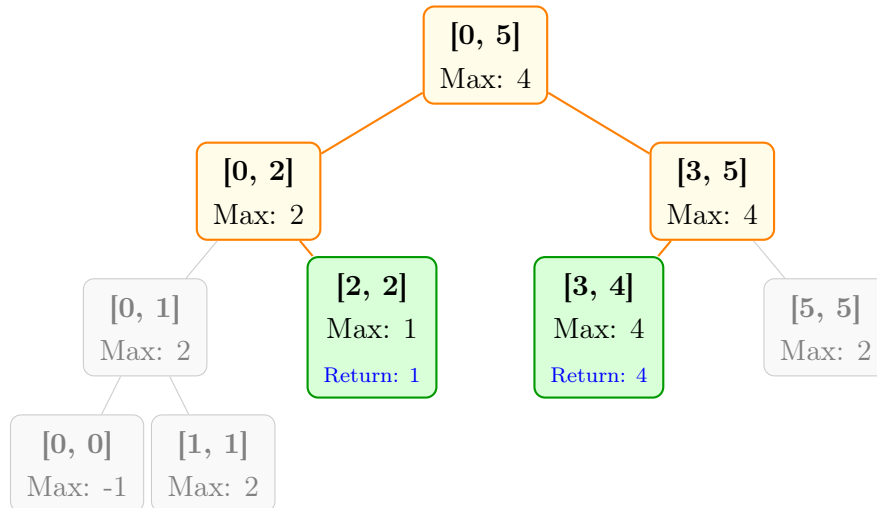


Figure 3: Execution trace of $\text{Max}(2, 4)$ after the update. Yellow nodes are explored to subdivide the query, while green nodes immediately return their value without exploring their children. The root node will finally calculate $\max(1, 4) = 4$.

The push-down logic of **lazy propagation** is therefore reapplied: if the `to_propagate` flag is active, the limit is applied to the children, the flag is passed to them, and the current node is "cleaned". This ensures that every query reads the correct state of the array without corrupting performance.

```
if self.tree[node].to_propagate {
    self.tree[node].to_propagate = false;

    self.tree[left_node].val = cmp::min(self.tree[node].val,
                                         self.tree[left_node].val);
    self.tree[left_node].to_propagate = true;

    self.tree[right_node].val = cmp::min(self.tree[node].val,
                                         self.tree[right_node].val);
    self.tree[right_node].to_propagate = true;
}
```

Once certain that the children possess the correct values, the recursive traversal continues only towards the branches that actually overlap with the search interval $[i, j]$. If a branch is completely outside the range of interest, it is not visited and returns a default neutral value for the maximum operation (in this case -1). Finally, the result for the current segment will simply be the maximum between the values returned by the left sub-tree and the right sub-tree.

```
let left = if i <= mid {
    self.max_rec(left_node, start, mid, i, j)
} else { -1 };

let right = if j > mid {
    self.max_rec(right_node, mid + 1, end, i, j)
} else { -1 };

return cmp::max(left, right);
```

Complexity Analysis

In terms of **Time Complexity**, all three fundamental operations (Insert, Update, and Max) operate in $\mathcal{O}(\log n)$ time. During range operations, thanks to *lazy propagation*, nodes completely included in the target range stop the immediate downward

propagation. Consequently, the number of visited nodes, where constant time $\mathcal{O}(1)$ operations are performed, is proportional to the tree's height. Accounting for the structural setup and m queries, the overall time complexity perfectly aligns with the target of $\mathcal{O}((n + m) \log n)$.

Regarding **Space Complexity**, the Segment Tree structure inherently requires an allocation of $\mathcal{O}(n)$ nodes. The additional temporary space required during the operations is dictated exclusively by the stack allocations necessary for the recursive function calls, whose maximum depth is equal to the tree's height, $\mathcal{O}(\log n)$. Therefore, the total space complexity is bounded by the tree's primary storage, resulting in $\mathcal{O}(n)$.

2 The "Is There" Problem

In this section, I will explain the solution to the "Is There" problem, designed to achieve a **Time Complexity** of $\mathcal{O}((n + m) \log n)$. Here, n corresponds to the array size (the points in the interval from 0 to $n - 1$) and m represents the number of queries.

To strictly meet the required time constraints, the solution is structured into three phases: calculating the segment coverages using a **Difference Array**, applying an offline processing approach based on the **Sweep-Line Algorithm**, and utilizing a **Segment Tree** to answer the queries.

1. Active Segments Calculation

First, we must determine exactly how many segments cover each position p (where $0 \leq p \leq n - 1$). To achieve this in $\mathcal{O}(n)$ time, we use the **Difference Array** technique. We create a vector `diff` of size $n + 1$. For each segment $\langle l, r \rangle$, we increment `diff[l]` by 1 and decrement `diff[r + 1]` by 1. Subsequently, by performing a *prefix sum* on this vector, we generate a `counts` array where each element `counts[p]` indicates exactly how many segments contain position p .

```
let mut diff = vec![0i32; n + 1];
for _ in 0..n {
    let l: usize = values_in.next().unwrap().parse().unwrap();
    let r: usize = values_in.next().unwrap().parse().unwrap();
    diff[l] += 1;
    diff[r + 1] -= 1;
}

// Prefix sum to obtain the coverage counts array
let mut counts = Vec::with_capacity(n);
let mut cur = 0i32;
for &d in diff[..n].iter() {
    cur += d;
    counts.push(cur);
}
```

2. Query Processing

The second step involves reading all the input queries and storing them organized by their right endpoint j . This phase prepares the data for the **Sweep-Line Al-**

gorithm. The core idea is to group the queries by j so that we can traverse the `counts` array linearly from left to right. By doing this, when our algorithm reaches position j , it will have already processed the entire sub-interval $0 \dots j$.

```
// We use a tuple (id, i, j, k) for each query
let mut queries_by_r: Vec<Vec<(usize, usize, usize, i32)>> =
    vec![Vec::new(); n];

for idx in 0..m {
    let i: usize = values_in.next().unwrap().parse().unwrap();
    let j: usize = values_in.next().unwrap().parse().unwrap();
    let k: i32 = values_in.next().unwrap().parse().unwrap();

    queries_by_r[j].push((idx, i, j, k));
}
```

3. Resolution via Segment Tree

The final phase combines the **Sweep-Line Algorithm** with the **Segment Tree**. In this architecture, the Segment Tree is **not** built over the positions of the original array, but rather **over the domain of possible k values** (which range from 0 to n). Each leaf node of the Segment Tree (representing a specific segment count k) stores the index p corresponding to the **last time** we encountered that exact number of segments.

Iterating through the `counts` array with a `pos` cursor from 0 to $n - 1$:

1. We update the Segment Tree: we take the value at `counts[pos]` and update the tree at the corresponding value's position, writing the current index `pos` into it.
2. Immediately after the update, we extract all the parked queries for the current position (`queries_by_r[pos]`).
3. To answer one of these queries (which asks if the value k exists between i and j), we query the Segment Tree for the value k . This returns `last_seen`, which is the most recent index where k was recorded.
4. If `last_seen` $\geq i$, it means the value k appeared within the desired range, so the query result is **1**. Otherwise, the value is not present, and the result is **0**.

```

let mut st = SegmentTree::new(n + 1);
let mut answers = vec![false; m];

for pos in 0..n {
    let val = counts[pos] as usize;

    st.update(1, 0, n, val, pos as i32);

    for q in &queries_by_r[pos] {
        let (q_id, q_i, _q_j, q_k) = *q;

        if q_k < 0 || (q_k as usize) > n {
            answers[q_id] = false;
        } else {
            let last_seen =
                st.query_point(1, 0, n, q_k as usize);
            answers[q_id] = last_seen >= (q_i as i32);
        }
    }
}

```

Complexity Analysis

In terms of **Time Complexity**, the initial setup using the *difference array* to calculate the coverages requires linear time, $\mathcal{O}(n)$. Grouping the queries into buckets takes $\mathcal{O}(m)$ time. During the sweep-line phase, we perform exactly n updates (one per position) and m point queries (one for each ‘IsThere’ query) on the Segment Tree. Because each Segment Tree operation traverses a root-to-leaf path—where the number of visited nodes is proportional to the tree’s height—each operation takes logarithmic time, $\mathcal{O}(\log n)$. Consequently, the total computational cost is dominated by the sweep-line phase, resulting in exactly $\mathcal{O}((n + m) \log n)$.

Regarding **Space Complexity**, the overall requirement is $\mathcal{O}(n + m)$. The Segment Tree, structured to accommodate values from 0 to n , requires a memory allocation of $\mathcal{O}(n)$ nodes. Storing the m queries in advance for offline processing via the *Sweep-Line* requires $\mathcal{O}(m)$ space. Finally, considering the operational overhead on the tree, the temporary auxiliary space is determined entirely by the stack allocations for recursive function calls. The maximum depth of these calls equals the tree’s height, $\mathcal{O}(\log n)$. However, this logarithmic overhead is strictly bounded and safely absorbed by the primary data structures’ sizes.